

EduQuest

Architecture Discovery & Reverse Engineering Report

Prepared For: Solution Architecture Review

Status: Completed (Database & App Migration Confirmed)

Date: July 3, 2026

EduQuest - Architecture Discovery & Reverse Engineering

This report documents the deep exploration and architectural mapping of the **EduQuest** gamified learning platform. It is designed to establish a perfect, zero-assumption baseline of the system's current architecture, business logic, flows, and dependencies.

Phase 1 - Repository Exploration

The EduQuest codebase is structured as a modern modular React project using TypeScript, Vite, and tailwind-style styling conventions. Below is the detailed folder hierarchy and directory responsibilities.

```

eduquest-main/
  src/
    assets/           # Static visual assets (illustrations, icons, local logos)
    components/       # Reusable presentation and layout components
      admin/          # Admin dashboard sub-views (StudentList, BulkImport, managers)
      adventure/      # Adventure Map and level map player logic
      layout/         # Base layout components (e.g. GameBackground)
      learning/       # Core student learning modules (AvatarShop, CharacterCreator, EnglishBuddy)
      marketing/      # Public landing page components
      ui/             # Core UI components (button, dialog, input, toasts)
    contexts/         # React context providers for global state
      AuthContext.tsx # User sessions, profile fetch, cache layers, roles
      ThemeContext.tsx# Color theme management (Dark mode / Light mode)
    data/             # Local static configurations (avatar cosmetics, quest configs)
    hooks/            # Custom React Hooks (useCompletionReward, useToast, etc.)
    i18n/             # Localization infrastructure (English, Tamil)
    integrations/     # Backend service SDK wrappers
      supabase/       # Supabase standard client, admin clients, auto-generated TS types
    locales/          # Translation dictionary resource files
    pages/            # Top-level route pages (lazy loaded)
    store/            # Client-side stores (e.g. state management hooks)
    styles/           # Global styles, animations, utility styling overrides
    test/            # Testing configurations and test helper scripts
    types/            # Domain-specific TypeScript declarations

```

Folder Roles and Dependencies

- ``contexts/AuthContext.tsx`` is the heart of global state, querying the database (``profiles``, ``user_roles``) to supply session parameters to ``components/ProtectedRoute.tsx`` which locks/unlocks page access.
- ``components/learning`` depends heavily on ``contexts/AuthContext.tsx`` for student coins, profile, and XP rewards, and invokes APIs via the Supabase client wrapper ``integrations/supabase/client.ts``.
- ``components/admin`` holds specialized management panels, using both standard clients and ``integrations/supabase/adminClient.ts`` to manage users and view tenant data.

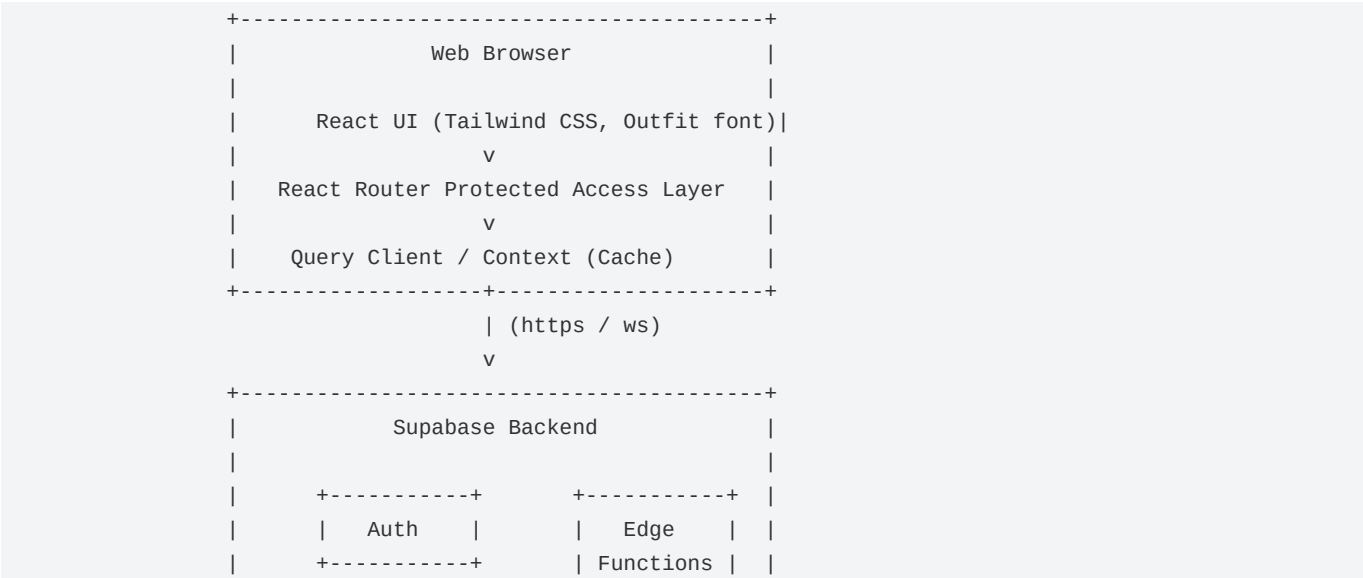
Phase 2 - Technology Stack Analysis

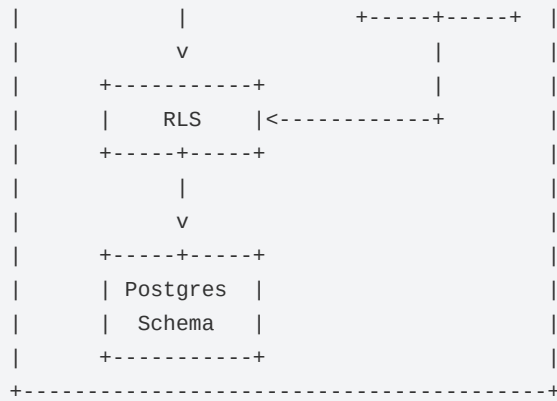
EduQuest is a highly interactive Single-Page Application (SPA) leveraging the following technologies:

---	---	---
Frontend Framework	React 18+ (TypeScript)	UI structure, component rendering, reactivity
Styling & Theme	Tailwind CSS + Vanilla CSS	Custom animations, gamified interfaces, dark-mode
Routing	React Router v6	Single Page client-side routing & Protected routes
State Management	React Context + React Query	Server-cache state management with automatic refer
Backend / DB	Supabase (PostgreSQL 15+)	Multi-tenant relational schema, functions, triggers, an
Authentication	Supabase Auth	Identity provider (Email/Password for admins; custom
AI Integration	Google Gemini API	Interactive AI English Buddy & Automated AI Quiz G
Build System	Vite + SWC compiler	Ultra-fast local compiling and code-split production b
Package Manager	npm	Local dependency management
Deployment	Vercel	Jamstack hosting with serverless route rewriting

Phase 3 - Application Architecture

Conceptual Diagram





Architectural Flow Explained

- 1. Routing & Guards:** Page access is checked using `ProtectedRoute``. If the user is a ``student``, they are restricted to ``/dashboard``, ``/quiz/:id``. If they are an ``admin``, ``super_admin``, or ``teacher``, they are allowed access to the ``admin`` workspace.
- 2. State & Cache:** Query client cache is configured with a stale-time of 30 seconds. To prevent auth lag, ``AuthContext`` caches the logged-in user profile in ``localStorage`` (``eq_user_data``), permitting instantaneous dashboard loads on revisit.
- 3. Database Security (RLS):** Row-Level Security ensures that students can only read and write their own data (``user_id = auth.uid()``), while administrators can read and manage data within their school (``school_id = get_user_school_id(auth.uid())``).

Phase 4 - Feature Discovery

Feature 1: Student Gamified Dashboard

- **Purpose:** Provide a portal for student learning, containing Streak indicators, Gem/Coin wallets, Level meters, and subject browsers.
- **Database Tables:** ``profiles``, ``student_progress``, ``subjects``, ``lessons``, ``quizzes``.
- **Components:** ``enhancedProfileCard``, ``DailyQuestSystem``, ``ContinueLearningCard``, ``GemWallet``.

Feature 2: Adventure Map

- **Purpose:** Provide a game-like level progression system. Students navigate maps to clear levels, earn stars, and collect rewards.
- **Database Tables:** ``adventure_progress``, ``profiles``, ``coin_transactions``.
- **Components:** ``AdventureMap``, ``AdventureLevelPlayer``.

Feature 3: Avatar Shop & Character Creator

- **Purpose:** Allow students to use earned coins to buy customize items (hairstyles, uniforms, fantasy gear, anime accessories) and equip them onto their custom avatar.

- **Database Tables:** `avatar\_items`, `student\_avatar\_items`, `profiles`, `coin\_transactions`.
- **Components:** `AvatarShop`, `CharacterCreator`.

Feature 4: AI English Buddy

- **Purpose:** Interactive speaking and conversation partner utilizing AI. Provides dialogue scripts, pronunciations, and corrections.
- **Database Tables:** `ai\_usage`.
- **APIs Used:** Gemini API via Edge Function.
- **Components:** `EnglishBuddy`.

Phase 5 - User Roles & Permissions

EduQuest features a structured 6-tier role matrix managed through the `user\_roles` table:

Role	Target	Access Level / Permissions
---	---	---
Platform Admin	SaaS Operator	Manage subscription plans, view all schools, edit global settings
Super Admin	School Owner	Complete management of their specific school: add/delete schools, manage users
School Admin	Operations Admin	View school analytics, create students, edit student profiles
Teacher	Learning Admin	View school progress, create quizzes, edit/add lessons
Student	Learner	Read-only lessons, take quizzes, buy avatar items, request feedback

Phase 6 - Navigation Flow

```
graph TD
    Landing[Landing Page /] -->|Student Login| SLogin[Student Login /login]
    Landing -->|Admin Login| ALogin[Admin Login /admin/login]
    Landing -->|Register School| Reg[Register School /register]

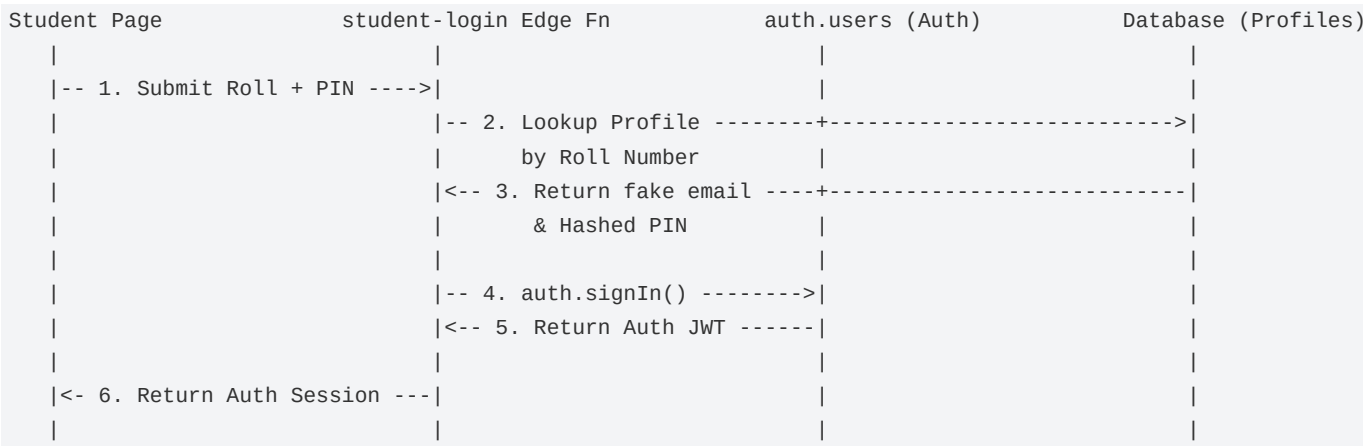
    SLogin -->|Authenticate| SDash[Student Dashboard /dashboard]
    SDash -->|Select Lesson| LessonView[Lesson Viewer /dashboard]
    SDash -->|Take Quiz| QuizPlay[Quiz Player /quiz/:id]
    SDash -->|Customize Character| CharCreator[Character Creator]
    SDash -->|Open Avatar Shop| Shop[Avatar Shop]

    ALogin -->|Authenticate| ADash[Admin Dashboard /admin]
    ADash -->|Manage Students| SList[Student List]
    ADash -->|Manage Lessons| LManager[Lesson Manager]
    ADash -->|View Analytics| Analytics[Analytics Panels]
```

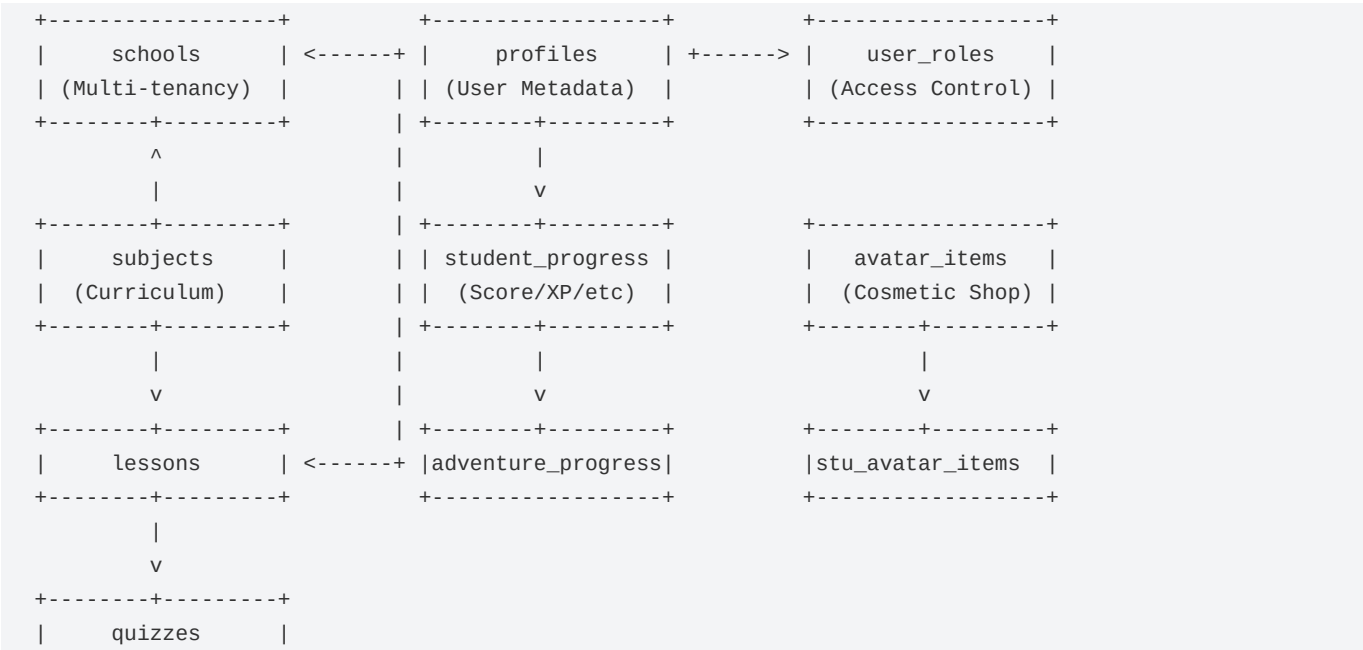
```
Reg -->|Setup School| ALogin
```

Phase 7 - Authentication Flow

Students log in via a custom, secure passwordless-style PIN flow implemented through Edge Functions to protect children's identities.



Phase 8 - Database Analysis



+-----+

Phase 9 - Storage Analysis

As confirmed by static analysis, EduQuest **does not use Supabase Storage buckets**. All avatar visual graphics, icons, badges, and cosmetics are stored locally inside the client code as static asset configuration mappings (`src/data/cosmetics.ts`, standard SVG assets) or fetch external URLs for profile photos. This simplifies deployment and keeps database footprint light.

Phase 10 - API Analysis (Edge Functions)

All edge functions communicate via HTTPS POST requests containing JWT tokens inside the `Authorization: Bearer <token>` header:

---	---	---	---
`/functions/v1/student-login`	Passwordless Student Auth	`{ rollNumber, pin }`	`{ session }` or `{ error }`
`/functions/v1/create-student`	Create Student (Admin Only)	`{ fullName, rollNumber, pin, classLevel }`	`{ success: true, userId }`
`/functions/v1/manage-teacher`	Create/Deactivate Teachers	`{ action: 'create'/'delete', email, fullName }`	`{ success: true }`
`/functions/v1/reset-student-pin`	Reset Student PIN (Admin)	`{ studentId, newPin }`	`{ success: true }`
`/functions/v1/generate-quiz`	AI Quiz Generator (Teacher/Admin)	`{ subject, topic, classLevel }`	`{ quiz: QuizObject }`

Phase 11 - Component Dependency Tree

```
App
  ThemeProvider & AuthProvider (Context wrappers)
    GameBackground
      BrowserRouter
        StudentLogin / AdminLogin / SchoolRegister
        ProtectedRoute
          StudentDashboard
            EnhancedProfileCard
            ContinueLearningCard
            SubjectBrowser
            DailyQuestSystem
            DailyLoginBonus
```

```

    EnhancedLeaderboard
    AvatarShop / CharacterCreator

    QuizPage
    QuizPlayer (contains Timer, QuestionViewer, QuizFeedback)

    AdminDashboard
    SchoolAnalytics
    ClassAnalytics
    StudentList
    AddStudentDialog & BulkImportDialog
    SubjectManager / LessonManager / QuizManager

```

Phase 12 - Business Logic Analysis

- XP & Level Scaling:** When a student completes a lesson or quiz, their score is evaluated. XP is credited to ``student_progress`` and ``coin_transactions``. The client calculates their current level dynamically using:
``Level = Math.floor(Sqrt(XP / 100)) + 1``
- Student Authentication Guard:** Because younger students are prone to forgetting email addresses, they sign in using a simple roll number (e.g. ``STU09``) and a 6-digit numeric PIN. The backend edge function constructs a deterministic email ``{rollNumber}@student.eduquest.local`` and logs them in silently.
- Multi-Tenancy Isolation:** To ensure complete safety in a SaaS school model, every query to metrics or user profiles filters by the school UUID (``school_id``).

Phase 13 - Configuration Analysis

- ``.env`` / **Environment Variables:**
- ``VITE_SUPABASE_URL``: Destination backend endpoint.
- ``VITE_SUPABASE_PUBLISHABLE_KEY``: Anon access key.
- ``SUPABASE_SERVICE_ROLE_KEY``: Secret admin override bypass key (Backend only).
- ``supabase/config.toml``: Instructs local Supabase emulator / deployment pipelines how to deploy the edge functions.

Phase 14 - Code Quality Review & Recommendations

- Materialized View Refreshes:** The ``school_analytics`` view compiles charts, but PostgreSQL materialized views do not refresh automatically. We should execute the ``refresh_school_analytics()`` cron trigger whenever student progress rows are updated to avoid stale dashboards.

2. **CORS Whitelist hardening:** Currently, Edge Functions whitelist all incoming localhost origins. In production, `ALLOWED\_ORIGIN` must be restricted strictly to your Vercel deployment URL.

Phase 15 - Feature Enhancement Readiness

Current Strengths

- Excellent UI/UX gamification layout with custom SVGs and sound effect compatibility.
- Safe, deterministic, passwordless student auth.
- High RLS coverage shielding cross-tenant breaches.

Recommended Next steps

1. Deploy any missing lesson databases.
2. Link school-specific parameters.
3. Hook up automated materialized view refresh.